

P51

SWE-bench: ¿pueden los modelos resolver incidencias reales de GitHub?

SWE-bench: Can Language Models Resolve Real-World GitHub Issues?

Carlos E. Jimenez, John Yang, Alexander Wettig, y otros · 2023 · arXiv:2310.06770 · ICLR 2024

Ficha pedagógica del Artificial Intelligence Evolution Program. No es el paper original: es una guía de lectura en español que enlaza a la fuente primaria. Consultado 2026-08-16.

P51 — SWE-bench

Evaluación y seguridad · Deja de preguntar si el código *parece* correcto. Pregunta si pasan los tests del repositorio.

Nivel: L3 · Motor: `swebench` · Notebook: `P51_swebench.ipynb` · Anexo: complejidad y coste

1. Identificación

Campo	Valor
Título original	<i>SWE-bench: Can Language Models Resolve Real-World GitHub Issues?</i>
Autoría	Carlos E. Jimenez, John Yang, Alexander Wettig y otros
Año	2023
Venue	arXiv:2310.06770 · ICLR 2024
Fuente primaria	arXiv:2310.06770
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Las evaluaciones de programación de la época —HumanEval, MBPP— pedían funciones autocontenidas de pocas líneas, con el enunciado en el propio docstring. Los modelos las saturaron rápido, y la distancia entre ese resultado y ser útil en un repositorio real seguía siendo enorme.

Lo que faltaba en esas pruebas es justo lo que define el trabajo real: **localizar** dónde tocar en un código que no cabe en el contexto, entender una incidencia escrita por una persona con información incompleta, y no romper nada más.

3. Propuesta

Construir la evaluación a partir de **incidencias reales resueltas** en repositorios de Python populares. Cada instancia contiene:

- el texto de la incidencia tal como lo escribió alguien;
- el repositorio en el commit exacto anterior a la solución;
- los **tests** que el arreglo real hizo pasar.

El criterio de éxito es binario y no negociable: aplicar el parche generado y ejecutar la suite. O pasan los tests, o no.

Ese criterio es lo que aporta el trabajo. Un test es un verificador objetivo que **no se puede convencer con prosa**.

4. Intuición sin fórmulas

Un examen de mecánica con dos correcciones posibles: describir cómo arreglarías el motor, o arrancarlo. La segunda no admite interpretación.

Dónde deja de funcionar la analogía: un motor que arranca funciona. Un test que pasa solo prueba lo que ese test comprueba — y puede haberse conseguido de una forma terrible.

5. Matemática mínima

% resuelto = instancias con TODOS los tests en verde / instancias totales

Cinco intentos, tres criterios:

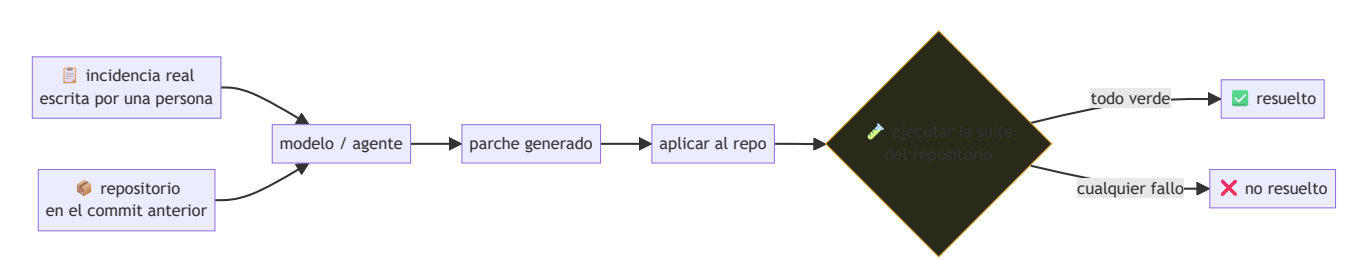
criterio	tasa
«parece correcto»	80 %
«compila»	80 %
tests pasan	40 %

La brecha entre 80 % y 40 % es exactamente el problema que ataca el benchmark. Los criterios blandos inflan, y siempre en la misma dirección.

[!TIP] **Puente matemático.** Esta sección da por sabido lo siguiente. Si algo no te suena, léelo primero: está explicado una sola vez, en un solo sitio, y sirve para todas las fichas.

Dónde	Qué necesitas de ahí
A02 §2 · Entropía	qué mide realmente una tasa de éxito y qué información aporta el criterio

6. Arquitectura o flujo



7. Qué observar en el paper original

- El **proceso de construcción**: cómo se filtran las incidencias para que sean verificables y reproducibles. Es la mitad del trabajo y casi nunca se comenta.
- Las **tasas iniciales**, muy bajas, y el contraste con los resultados saturados de HumanEval.
- La discusión sobre **contaminación**: las incidencias son públicas y anteriores al corte de datos de los modelos evaluados. El paper es honesto al respecto.
- El subconjunto **SWE-bench Lite** y por qué existe: el coste de ejecutar la evaluación completa no es trivial.

8. Evidencia y resultados

Evaluación de modelos de lenguaje sobre más de dos mil incidencias reales de doce repositorios de Python, con tasas de resolución iniciales de un solo dígito porcentual.

Las cifras están en el artículo, y **envejecen muy rápido**: la tasa del estado del arte ha subido mucho desde 2023. Verificar siempre contra la tabla pública vigente, no contra una cifra citada de memoria.

La miniatura de este eje no evalúa nada: con cinco casos escritos a mano, exhibe la diferencia entre medir apariencia y medir tests.

9. Impacto

- Se convirtió en la referencia de facto para agentes de programación, y en el número que se cita al presentar cada modelo nuevo.
- Impulsó la investigación en **localización de código**: encontrar dónde tocar resultó ser tan difícil como escribir el arreglo.
- Estableció un patrón para el resto del campo: evaluar con verificadores objetivos y ejecutables siempre que exista uno.
- Y dejó ver un límite del enfoque: cuando un benchmark se vuelve el objetivo, empieza a optimizarse para él.

10. Limitaciones

1. **Contaminación**: las incidencias y sus soluciones son públicas y pueden estar en los datos de entrenamiento.
2. **Pasar los tests no es una buena solución**: puede lograrse de forma frágil, rompiendo el diseño, o incluso modificando los tests si no se impide.
3. **Solo Python**, y solo repositorios con buena cobertura de tests: un sesgo importante.
4. **Muchas incidencias reales no tienen test asociado**, y quedan fuera por construcción.
5. **Coste de ejecución alto**, lo que empuja a evaluar en subconjuntos y complica comparar cifras.
6. **Optimizar para el benchmark** es un riesgo real desde que se convirtió en la métrica pública de referencia.

11. Errores comunes

Error	Corrección
«Resuelve el X % de incidencias reales»	El X % de estas incidencias, de repos con buenos tests, en Python. La generalización es una afirmación aparte.
«Pasar los tests es resolver bien»	Es un mínimo objetivo, no un juicio de calidad. Nada dice del diseño ni del mantenimiento.
«Comparar cifras entre informes»	Hay variantes (completo, Lite, Verified) y andamiajes distintos. Sin especificar cuál, las cifras no son comparables.
«La contaminación está descartada»	No lo está, y el propio paper lo discute. Es una limitación estructural del diseño.
«Es la mejor medida de un programador»	Mide reparación de incidencias con test. No mide diseño, revisión, comunicación ni trabajo en un código sin cobertura.

12. Relación con trabajos anteriores

- **HumanEval (2021)** — la evaluación que satura y que este trabajo reemplaza. [arXiv:2107.03374](#)
- **P13 ReAct (2022)** — el patrón de agente que se evalúa aquí.
- **P16 Sistemas agénticos** — los sistemas que este benchmark mide.

13. Relación con trabajos posteriores

- **SWE-agent (2024)** — el andamiaje de interfaz agente-computadora construido sobre este benchmark. [arXiv:2405.15793](#)
- **SWE-bench Verified (2024)** — subconjunto revisado por personas para eliminar instancias mal especificadas.
- **Benchmarks ejecutables en otros dominios (2024+)** — la generalización del criterio.
- **P50 IA constitucional (2022)** — el enfoque opuesto: criterios de juicio donde no hay verificador objetivo posible.

14. Notebook asociado

`P51_swebench.ipynb`

Qué implementa: cinco intentos con tres criterios de éxito distintos y el cálculo de la tasa según cada uno, para hacer visible cuánto inflan los criterios blandos.

Qué NO implementa: no hay repositorio, ni incidencia, ni parche, ni suite de tests. Cinco casos escritos a mano no son una evaluación: son la ilustración de un criterio.

```
ai-evolution paper-lab P51 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Enumera los tres componentes de una instancia del benchmark.
Explicar	Explica por qué un test es mejor verificador que un juicio.
Aplicar	Ejecuta el notebook y añade un caso que compile pero rompa otro test.
Analizar	¿Por qué la contaminación es un problema estructural aquí?
Evaluar	Dos informes reportan cifras distintas. ¿Qué preguntas antes de compararlas?
Crear	Diseña un benchmark ejecutable para una tarea de tu dominio.

16. Autoevaluación

1. ¿Qué contiene cada instancia?
2. ¿Cuál es el criterio de éxito y por qué es distinto de los anteriores?
3. ¿Qué muestra la brecha entre 80 % y 40 %?
4. ¿Por qué la contaminación es difícil de descartar?
5. ¿Qué no mide este benchmark?
6. ¿Por qué las cifras entre informes no son directamente comparables?
7. ¿Qué habilidad resultó ser más difícil de lo esperado?

17. Respuestas esperadas

1. El texto de una incidencia real, el repositorio en el commit anterior a su solución, y los tests que el arreglo real hizo pasar.
2. Aplicar el parche y que **todos** los tests pasen. Es objetivo y ejecutable, frente a criterios de similitud con una solución de referencia o de juicio sobre el código.
3. Cuánto inflan los criterios blandos: el doble de tasa aparente frente a la verificable, siempre en la dirección optimista.
4. Porque las incidencias y sus soluciones son públicas en GitHub y anteriores al corte de datos de los modelos evaluados. No hay forma limpia de garantizar que no estuvieran en el entrenamiento.
5. Diseño, mantenibilidad, revisión, comunicación, y el trabajo en repositorios sin cobertura de tests. Tampoco cubre lenguajes distintos de Python.
6. Porque existen variantes del conjunto (completo, Lite, Verified) y andamiajes de agente muy distintos. Sin especificar ambos, los números miden cosas diferentes.
7. Localizar dónde hay que tocar. En un repositorio que no cabe en el contexto, encontrar el archivo y la función resultó tan difícil como escribir el arreglo.

18. Fuentes primarias

- Jimenez, C. E. et al. (2024). *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* **ICLR 2024**. arXiv:2310.06770 · consultado 2026-08-16.
- Yang, J. et al. (2024). *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. arXiv:2405.15793 · consultado 2026-08-16.



Evaluación — P51 · SWE-bench: ¿pueden los modelos resolver incidencias reales de GitHub?

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* (2023, arXiv:2310.06770 · ICLR 2024) · **Nivel:** L3

Parte A — Contexto histórico (20 pts)

- (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
- (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

- (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
- (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

- (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

- (10) Ejecuta `P51_swebench.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
- (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
- (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

- (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
- (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).